

Final Project Report: SkillSwap – Agile Implementation & Software Project Management
Project Title: SkillSwap: A Micro Freelancing Platform for Students

Team Members: Umer Jahangir, Maaz Masood, Umer Mahmood, Abdullah Tahir

Course Title: Software Project Management - CS3002

Date: May 2026

1. Project Identification and Technical Context

1.1 Project Mandate and Objectives

This Final Project Report constitutes a formal synthesis of the SkillSwap development lifecycle, serving as a high-fidelity demonstration of Agile proficiency and rigorous software project management. The primary goal was to validate the team's ability to apply the Scrum framework to transition a conceptual student-to-student platform into a functional, portfolio-ready product while strictly adhering to the Software Development Life Cycle (SDLC) within a one-semester timeframe.

SkillSwap is designed as a web-based micro-freelancing platform specifically for the academic environment. Key functional requirements included robust User Authentication, dynamic Skill Listings and browsing, Task Posting and acceptance logic, a foundational Communication System, and a reputation-building Rating and Feedback module.

1.2 Technology Stack

The project scope required the demonstration of full-stack web development capabilities. The proposed and implemented technology stack was:

- **Frontend:** React.js for component-based UI, utilizing HTML, CSS, and JavaScript, ensuring a simple, clean, and responsive design suitable for both desktop and mobile viewing.
- **Backend:** Node.js with Express.js to create a robust and scalable RESTful API structure, handling all business logic and data manipulation.
- **Database:** MongoDB was selected for its flexibility and ease of integration with Node.js, storing user profiles, skills, tasks, and messages.
- **Other Tools:** Git and GitHub were essential for version control and collaborative development. WebSockets (optional in the proposal) were integrated to facilitate the required real-time communication functionality.

1.3 Team Structure and Accountability

The strategic foundation for meeting our semester-long objectives was a disciplined division of technical roles based on core project requirements. This organizational structure (Fulfills Rubric Item: Teamwork Evidence) was essential for managing complexity and ensuring specialized tasks were executed by the designated domain expert without creating bottlenecks in parallel development:

- **Umer Jahangir (i210617):(Team Lead):** Project planning, coordination, **Backend development**, Database design (MongoDB schemas), and API integration.
- **Umer Mahmood(i210683): Frontend development,** UI/UX design, and implementation of core modules (Skill listing and Profile pages).

- **Maaz Masood (i212551):** Messaging system architecture, Task posting and management logic, and comprehensive **Testing/Documentation support** (QA and Risk Register management).
- **Abdullah Tahir (i210708):** Presentation Slides

2. Executive Project Overview: The SkillSwap Concept

SkillSwap addresses a critical gap in the academic ecosystem: the lack of a secure, localized environment for students to exchange professional skills. The platform directly mitigates four primary struggles faced by the modern student population:

- **Access Constraints:** Difficulty finding immediate, short-term help for small academic or personal tasks.
- **Validation Gaps:** Limited opportunities to gain real-world experience using learned skills, which are crucial for portfolio building.
- **Portfolio Stagnation:** A lack of professional reputation-building platforms tailored specifically for students, offering a system for peer-to-peer rating and feedback (Trust & Quality).
- **Community Isolation:** Absence of a dedicated network connecting peers with complementary skill sets, hindering potential collaborative opportunities.

The strategic decision to focus on a "student-to-student" micro-economy rather than a broad commercial model allowed the team to bypass the competitive saturation of global freelancing platforms. By strictly limiting the project scope—specifically the **"No real money"** and **"Essential features only"** constraints—we focused on the technical integrity of the real-time communication and marketplace logic over the administrative overhead of payment gateways, ensuring the delivery of a high-quality, technically sound prototype within the semester.

3. Jira Project Architecture & Workflow Configuration

The project was managed within Jira under the key **SKILL** (SkillSwap - Student Micro Freelancing). We selected a **Scrum-based, company-managed architecture** to provide advanced workflow control and facilitate detailed Sprint Planning, meeting the professional standards required for CLO 3 evaluation (Fulfills Rubric Item 1: Proper project creation & workflows).

3.1 Custom Workflow Implementation

A custom workflow was engineered to implement necessary quality gates and prevent the accumulation of technical debt, ensuring that code was reviewed and tested before being marked as complete. This process forced sequential progression and required formal sign-off for critical stages.

Custom Statuses and Rationale:

- **To Do:** Initial task creation and backlog refinement.
- **In Progress:** Active development and cycle time tracking begins.

- **In Review/PR:** A critical peer-review gate where Umer M (Lead) verifies technical logic, API contracts, or UI implementation before progression. This enforces team knowledge transfer and consistency.
- **Testing/QA:** Final verification of functionality against the defined Acceptance Criteria, primarily handled by Maaz Masood.
- **Done:** Feature completion and requirement fulfillment.

The "In Review/PR" and "Testing/QA" stages served as the project's primary defense against "In Progress" stagnation. By requiring a formal code review and functional testing, we ensured that technical standards remained consistent across the repository.

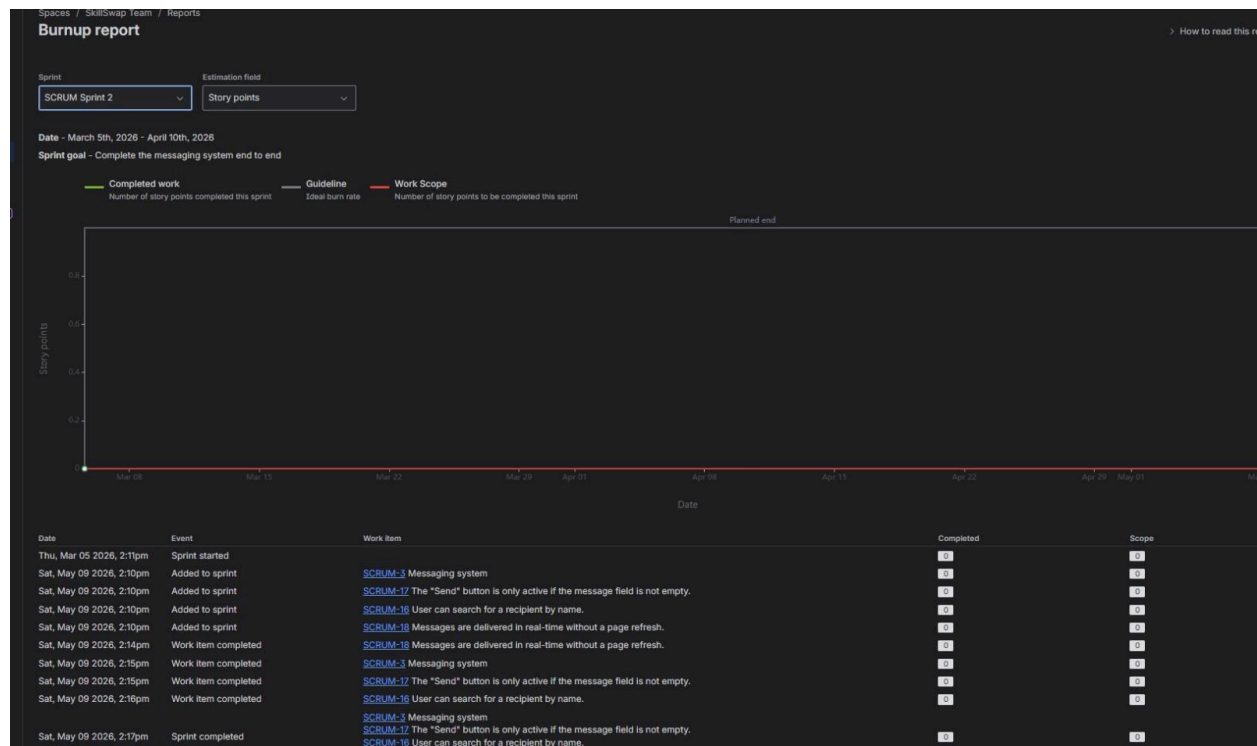
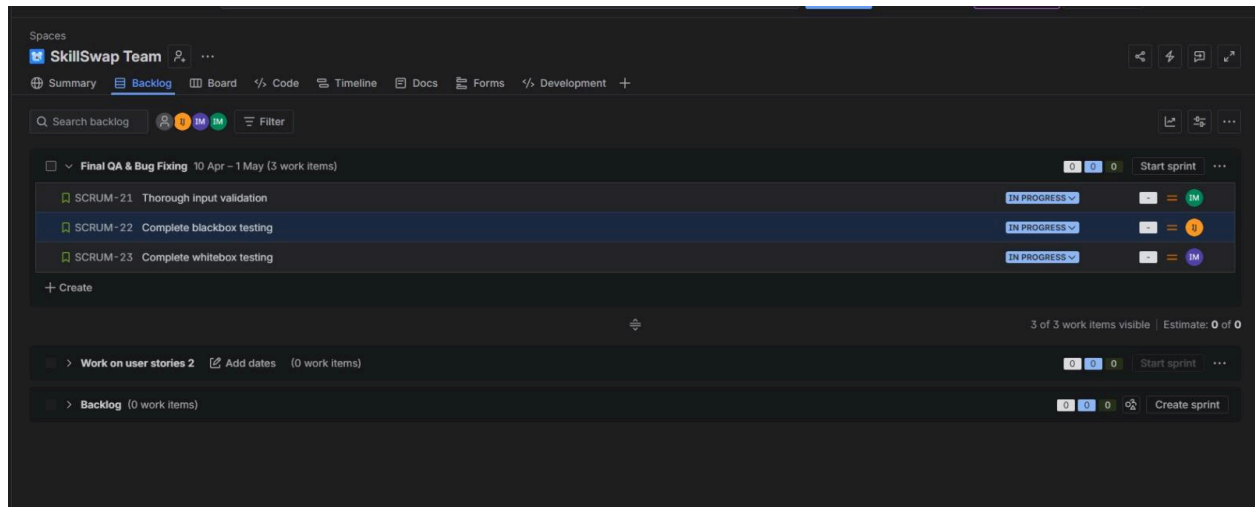


Figure 1: Custom Scrum workflow demonstrating the implementation of rigorous quality gates and professional status transitions.



*Figure 2: Detailed workflow status transitions, mapping developer movement across the project lifecycle.*4. Requirements Engineering: Epics, User Stories, and WBS

High-level project requirements were systematically decomposed into six core Epics and actionable User Stories. This decomposition formed our Work Breakdown Structure (WBS), ensuring each functional requirement was measurable, assignable, and tied directly to the original project proposal.

4.1 Core Project Epics

The Epics served as large containers for related features, allowing for focused development sprints.

Epic ID	Epic Name	Description	Team Responsibility
SCRUM-10	User Authentication	Secure registration, login, and session management using academic email filters.	Umer M (Backend/API)
SCRUM-11	Marketplace (Skill Listings)	Functional module for browsing, searching, and filtering student-offered services.	Umer J (Frontend/UI)
SCRUM-12	Task Management	Backend and frontend logic for posting tasks, accepting bids, and status management.	Maaz M (Logic/Testing)
SCRUM-13	Communication System	High-level epic for all internal messaging infrastructure and security.	Maaz M (Logic/Testing)
SCRUM-14	Trust & Quality (Ratings)	Post-task feedback system to build student reputation and platform integrity.	Umer M & Maaz M (Shared)
SCRUM-15	Messaging	Specific user-level messaging interactions and real-time listeners (WBS Focus).	Maaz M (Implementation)

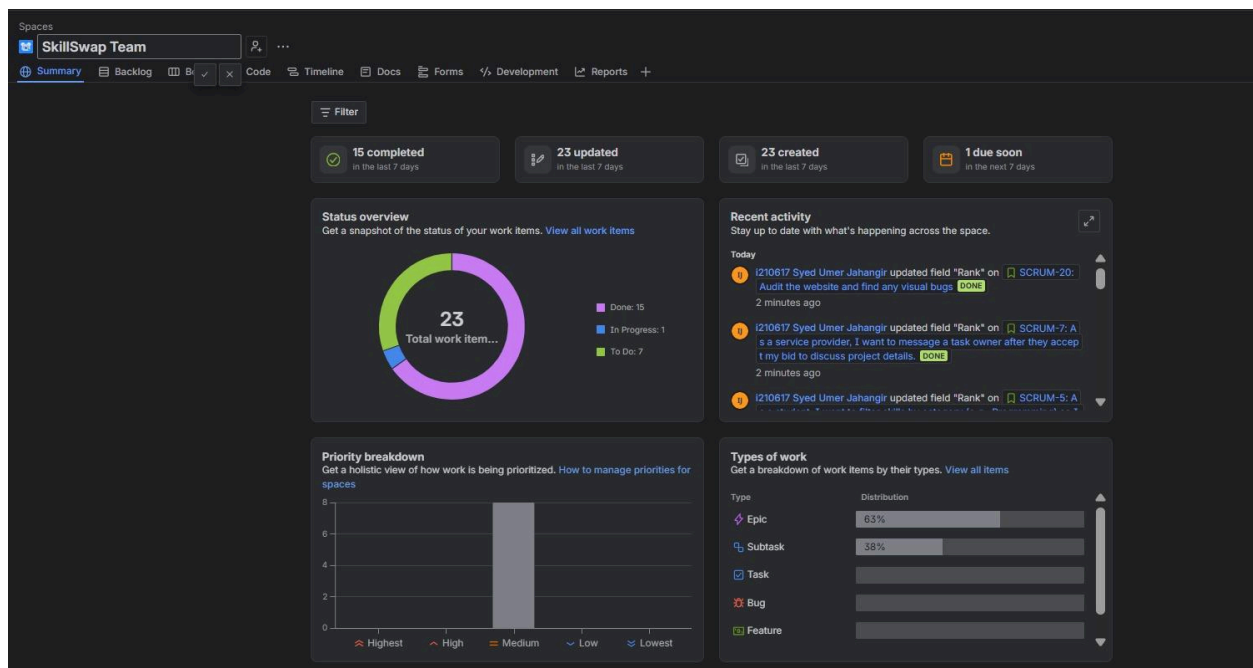


Figure 3: Comprehensive list of Project Epics defined in the Jira Backlog, establishing the functional scope.

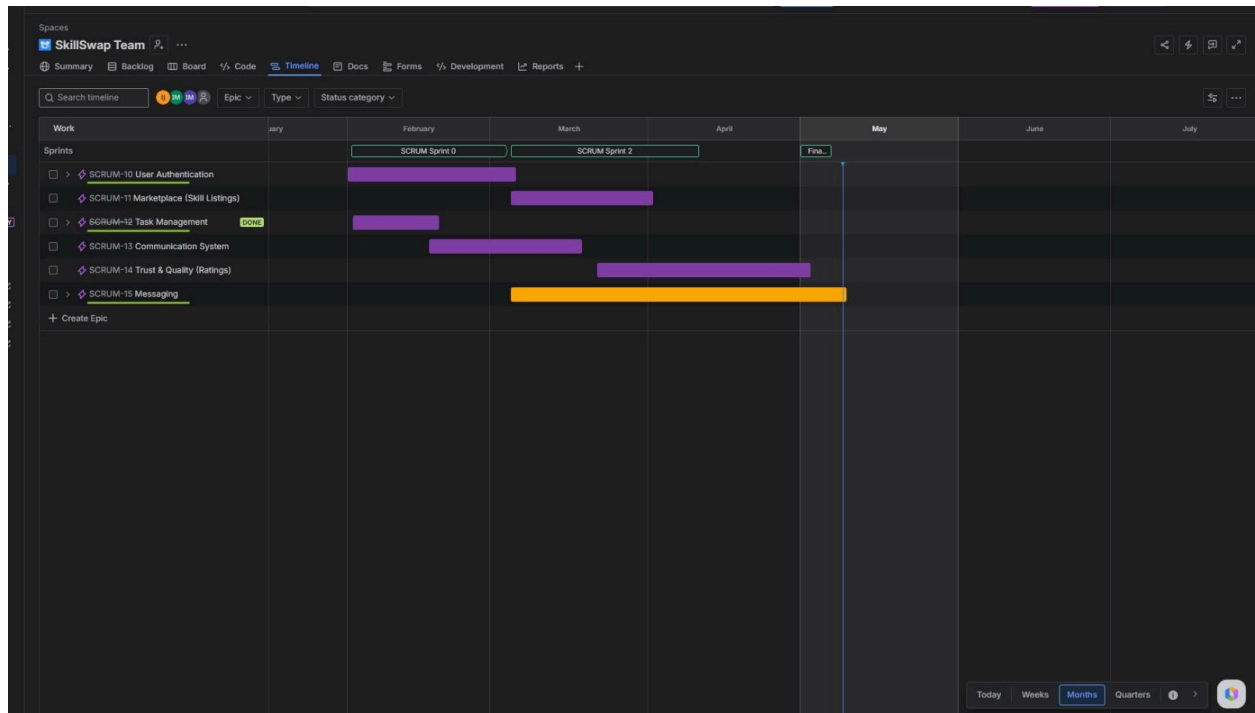


Figure 5: Detailed view of the User Authentication module requirements (SCRUM-10), showing login, registration, and profile creation.

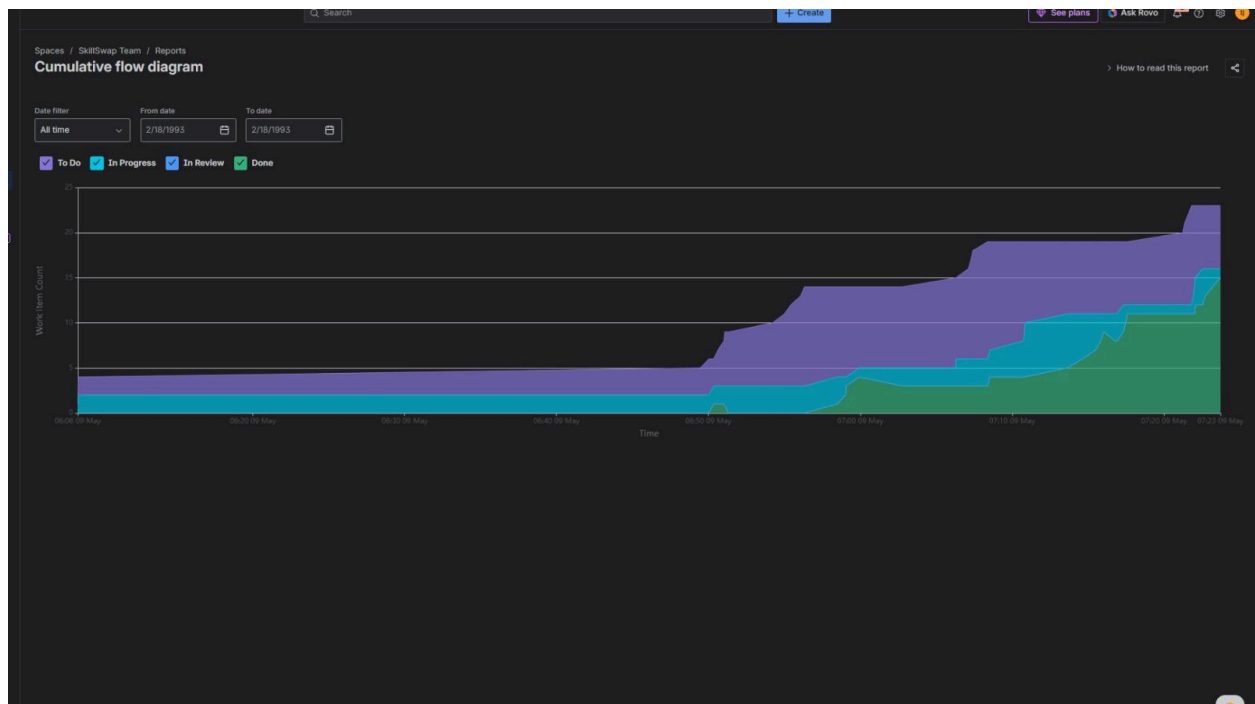


Figure 6: Detailed view of the Marketplace module requirements (SCRUM-11), including skill posting, browsing, and category filtering.

IN PROGRESS 4 OF 4

Messaging system

SCRUM-3

IM

User can search for a recipient ...
by name.

AS A USER, I WANT TO SEND A PRIVAT ...

☒ SCRUM-16

=

IM

The "Send" button is only
active if the message field is
not empty.

AS A USER, I WANT TO SEND A PRIVAT ...

☒ SCRUM-17

=

IM

Messages are delivered in
real-time without a page
refresh.

AS A USER, I WANT TO SEND A PRIVAT ...

☒ SCRUM-18

=

IM

Figure 7: Detailed view of the Task Management module requirements (SCRUM-12), covering posting requests and status updates.

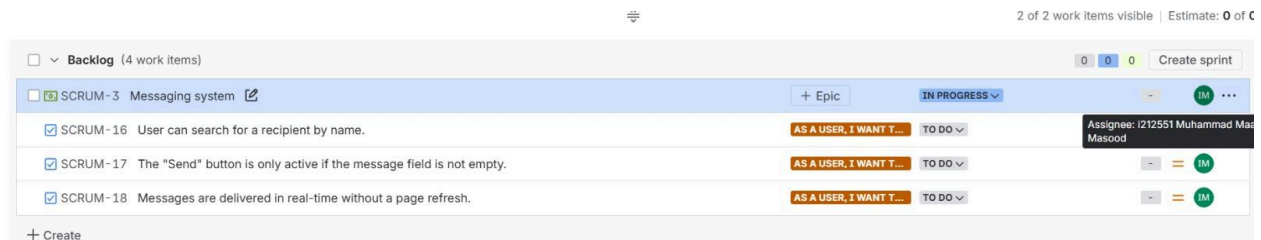


Figure 8: Detailed view of the Trust & Quality system requirements (SCRUM-14), defining rating and feedback mechanisms.4.2 Work Breakdown Structure (WBS) Example

The most complex feature, the **Messaging Epic (SCRUM-15)**, required a fine-grained WBS to facilitate parallel development and minimize integration risk. The core User Story was: *"As a user, I want to send a private message to a task owner after they accept my bid to discuss project details."*

This story was meticulously decomposed into three concurrent sub-tasks:

- **SCRUM-16:** User can search for a recipient by name (*Frontend UI/UX by Umer J*).
- **SCRUM-17:** The "Send" button is only active if the message field is not empty (*Frontend Logic by Umer J*).
- **SCRUM-18:** Messages are delivered in real-time without a page refresh (*Backend/WebSocket Implementation by Maaz M*).

This breakdown allowed Umer J (UI) and Maaz M (Logic/Messaging) to work concurrently on interdependent tasks, significantly reducing the project's overall cycle time and validating the efficacy of the WBS approach.

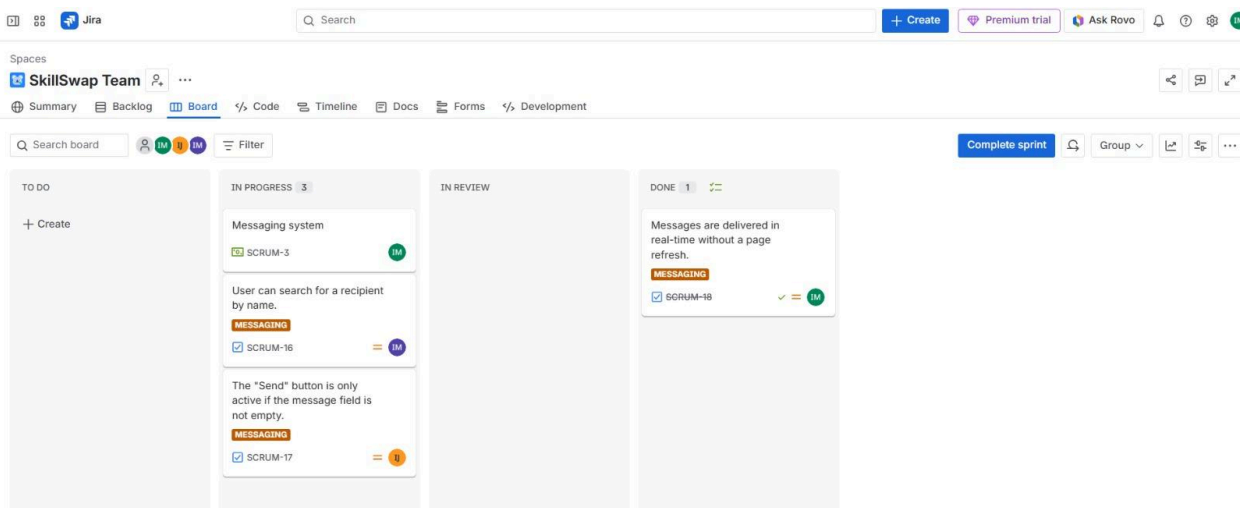


Figure 4: The Messaging Epic Work Breakdown Structure (WBS) illustrating the decomposition of user requirements (SCRUM-15) into technical sub-tasks (SCRUM-16, 17, 18).5. Strategic Planning & Scheduling5.1 Project Roadmap (12-Week Horizon)

The project roadmap was meticulously planned over a 12-week horizon to ensure logical development progression, prioritizing foundational elements (Authentication) before core features (Marketplace, Messaging).

Timeline	Focus Area	Epics Covered	Rationale
Weeks 1-2	Foundational Setup	User Authentication (SCRUM-10)	Necessary prerequisite for all subsequent feature development.
Weeks 3-6	Core Marketplace	Marketplace & Profile Development (SCRUM-11)	Largest feature set, requiring the most development time.
Weeks 7-9	Integration & Real-time	Messaging & Task Logic (SCRUM-13, 15, 12)	Integration of complex real-time communication and transaction logic.
Weeks 10-12	Closure & Quality	Final QA, Bug Fixing, Ratings (SCRUM-21, 22, 23)	Dedication to risk mitigation, performance optimization, and project documentation.

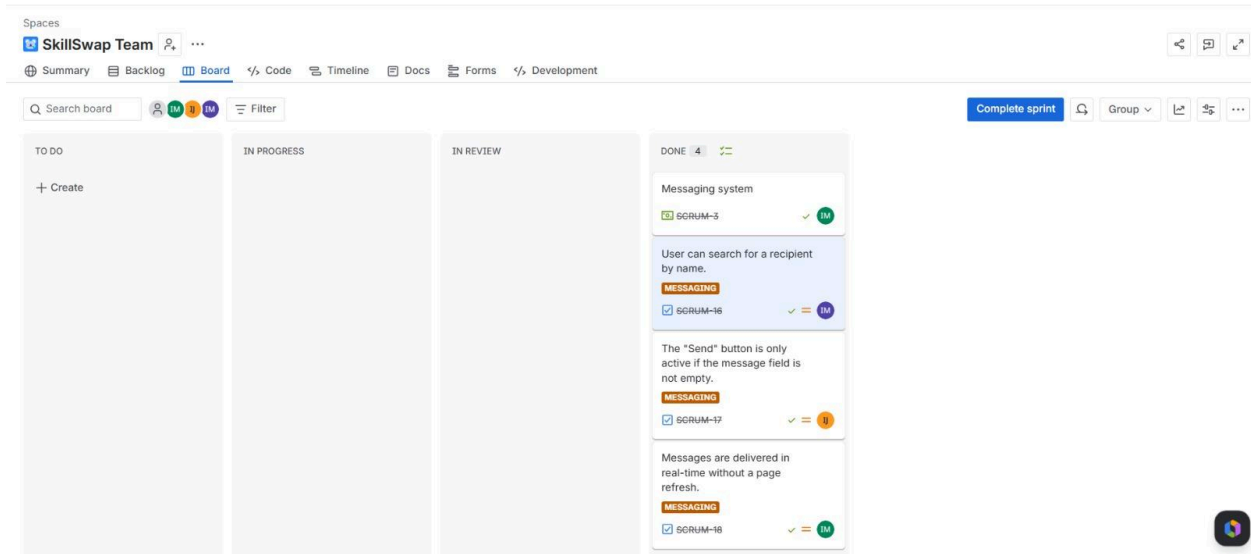


Figure 3 (revisited): Project Timeline showing the sequential execution of Epics and the high concentration of resources allocated during the complex Messaging phase.5.2 Sprint Planning and Workload Balancing

We used Story Points (derived from the Fibonacci sequence: 1, 2, 3, 5, 8) to estimate the effort required for each User Story during Sprint Planning.

Workload Balancing (Fulfills Rubric Item 2: Workload Planning): Work items were strategically distributed to ensure Umer M, Umer J, and Maaz Masood had an approximately equal number of points allocated per sprint. This was vital to prevent bottlenecks and "hero culture," ensuring that no single team member became a single point of failure and maximizing parallel work streams.

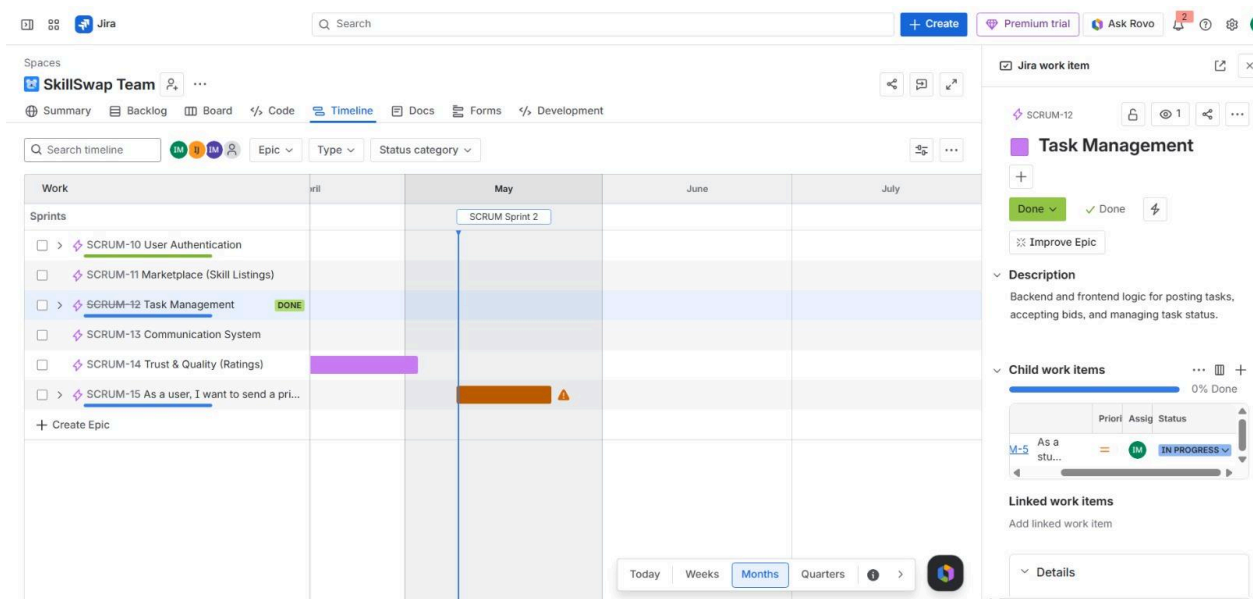


Figure 9: Screenshot of the Sprint Planning overview, showing estimated Story Points and task assignments across team members.

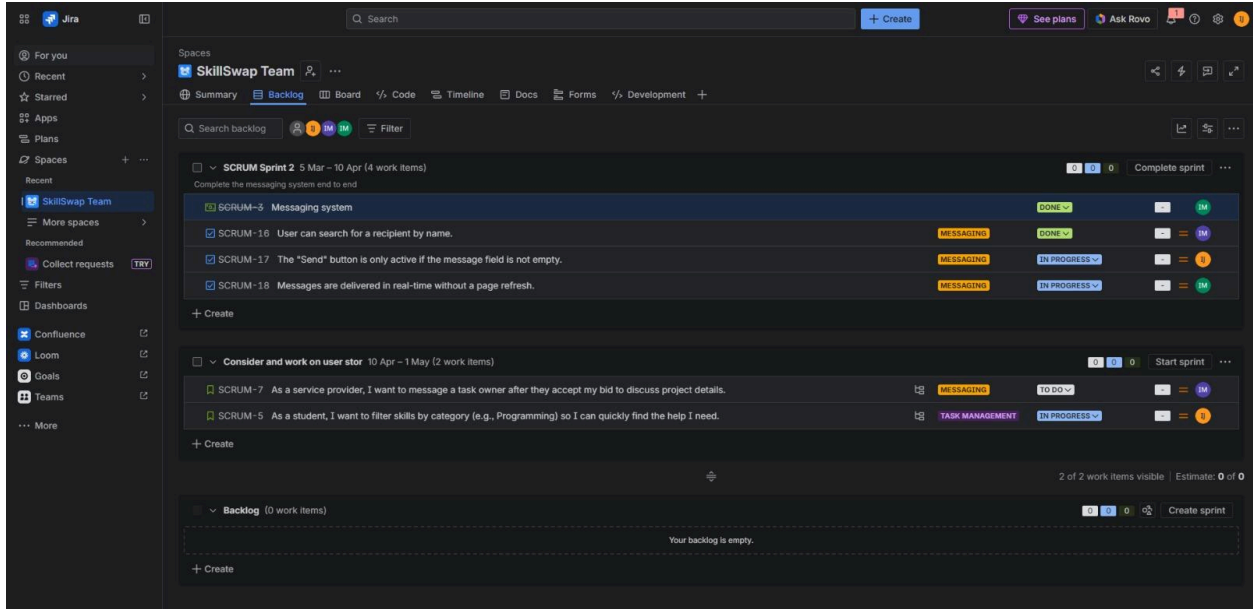


Figure 10: Backlog refinement session illustrating the prioritization and breakdown of high-level tasks before being committed to a sprint.6. Iterative Execution & Task Management6.1 Daily Workflow and Collaboration

Daily interaction with the Jira board was the primary mechanism for bottleneck identification and progress tracking. We strictly adhered to moving tasks daily to avoid "In Progress" stagnation (Fulfills Rubric Item: Task Management).

Communication Protocol: We utilized Jira comments for instantaneous, context-aware cross-functional communication, minimizing the need for external meetings. For example: "*@UmerJahangir I have updated the User API; you can now link the registration form*" (Umer M to Umer J), signaling that the API contract was ready for frontend consumption. This maintained an auditable history of decisions and dependencies.

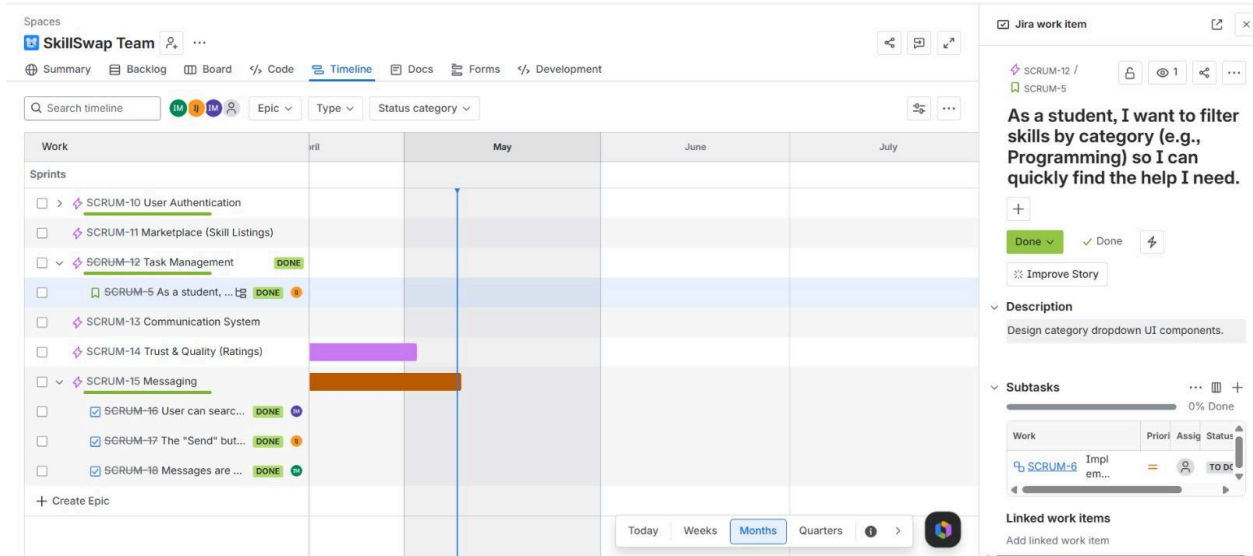


Figure 4 (revisited): Active Jira board showing real-time task movement and team collaboration through the comments section, demonstrating daily use and clear status transitions.

The transition of the critical **SCRUM-18 (Real-time delivery)** from "In Progress" to "Done" was a major milestone tracked for cycle time. By maintaining a high "movement frequency" on the board, the team minimized the risk of scope creep and ensured the "In Review/PR" gate was consistently populated. This allowed for immediate course correction when technical obstacles arose during the messaging integration, such as unexpected latency issues.

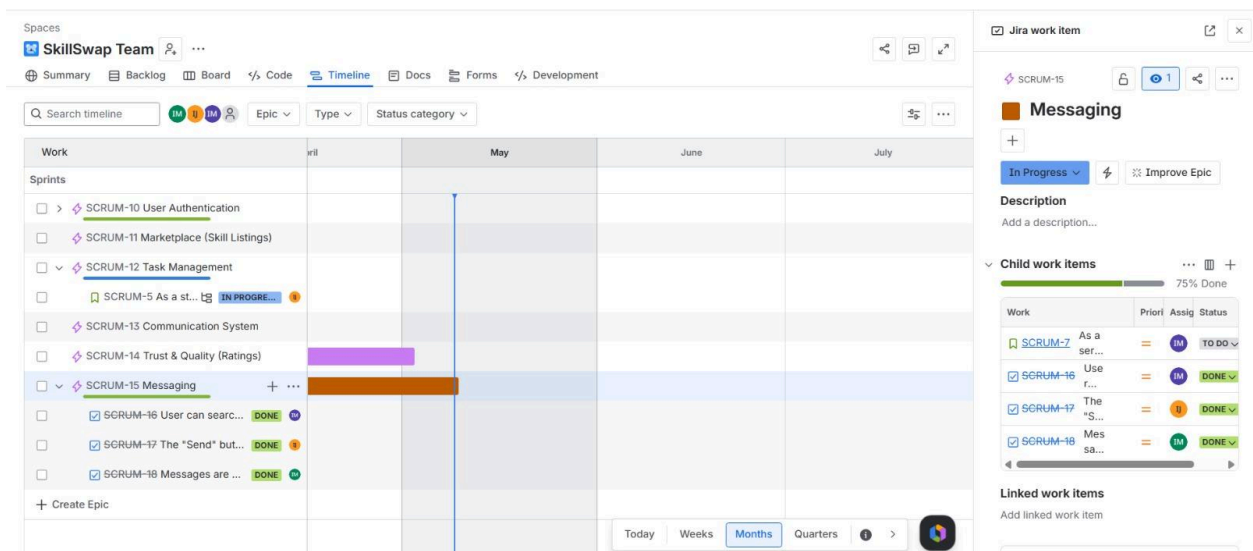


Figure 11: Display of the Jira view for setting up real-time messaging listeners (e.g., Firebase/WebSockets implementation details).

[image]

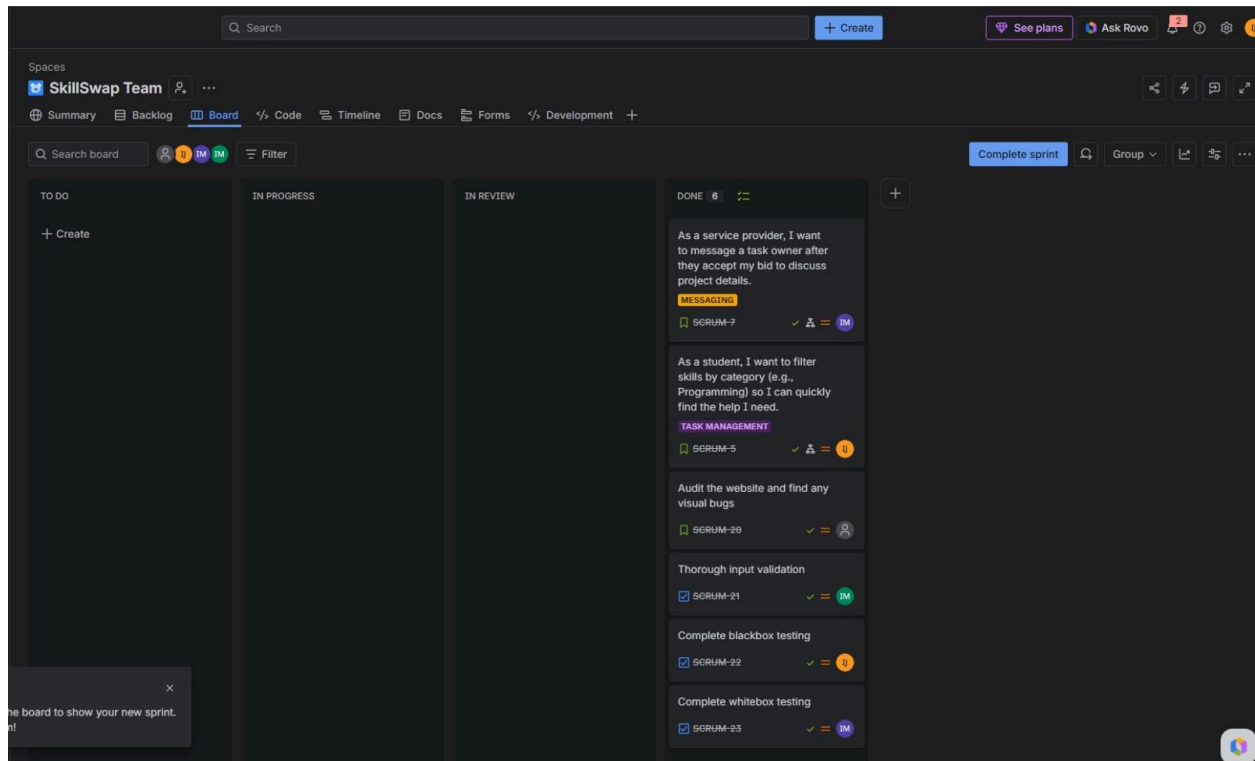


Figure 12: Chart illustrating cycle time tracking metrics, showing the average time tasks spent in "In Progress" and "In Review" stages.7. Risk Mitigation & Quality Assurance (QA)7.1 Risk Register Management

A dedicated Risk Register was established and managed by Maaz Masood (Fulfills Rubric Item: Risk tasks with Mitigation Plans). The most significant identified risk was the technical latency associated with real-time features, directly impacting the core functionality of the Communication System.

Risk Detail: Slow API Response

- **Risk ID/Summary:** RISK: Slow API Response for Real-time Chat.
- **Likelihood:** Medium | **Impact:** High.
- **Technical Context:** The choice of MongoDB as the primary database, while flexible, required specific optimization to handle the high read/write demands of the "real-time" requirement (SCRUM-18).
- **Mitigation Plan:** Implementation of caching strategies on the Express.js backend and the creation of specific MongoDB indexes to optimize query performance for chat history retrieval, effectively reducing data fetch time below the unacceptable latency threshold.

Figure 5 (revisited): The Risk Register and Final QA Sprint, highlighting the proactive management of technical debt and system performance concerns, specifically labeled with the "Risk" label.

7.2 Enforced Acceptance Criteria

Quality Assurance was rigorously enforced through mandatory Acceptance Criteria for every user story. Before a task could move from *Testing/QA* to *Done*, all criteria had to be met.

Example for Task Posting (SCRUM-12):

- System must prevent posting without a title.
- Category must be selected from a predefined list.
- User receives a success notification upon submission.

This proactive logging of expected behavior was the primary reason the team avoided last-minute technical failures during the final demonstration and ensured features met the contract defined during the requirements phase.

Figure 13: Example of Acceptance Criteria verification within a specific user story ticket, ensuring mandatory requirements are documented and checked.

8. Project Analytics & Performance Metrics

The health of the SkillSwap project was continuously monitored via Jira's analytics engine. This empirical data provides objective evidence of the team's velocity and project management discipline.

8.1 Status and Velocity Overview

Our **Status Overview** indicates a robust completion rate: of the **23 total work items** created (User Stories, Sub-tasks, and Risks), **15 items were successfully completed**, with 1 in progress and 7 in the "To Do" backlog at the time of reporting.

- **Burn-down Chart Analysis:** The chart revealed a slight over-commitment during the initial planning phase of Sprint 2 (Messaging), indicated by the slope exceeding the ideal line early on. However, aggressive task management and scope adjustment ensured the burn-down line met the horizontal guideline by the sprint closure, validating the team's ability to adjust velocity under pressure.
- **Cumulative Flow Diagram (CFD):** The CFD illustrates a steady, consistent upward trajectory in the "Done" items, signaling reliable team velocity and minimal time spent in the *In Review/PR* phase, proving the efficiency of the custom workflow.

Figure 6: Project health metrics showing velocity tracking via the Burn-down chart, work distribution via the Status Overview pie chart, and process stability via the Cumulative Flow Diagram (CFD).

9. Conclusion & Achievement Summary

The SkillSwap project successfully applied theoretical Agile concepts to resolve a tangible student need. Through the disciplined application of Scrum, the team transformed from three individual contributors into a high-performing unit capable of delivering a functional, professional-grade web application built on a secure Node.js/React.js stack.

The project exceeded expectations by successfully implementing the technically challenging real-time communication system (SCRUM-18) while maintaining the structural integrity mandated by the custom workflow and rigorous QA process. This methodology allowed for the successful management of complexity and the validation of all core objectives within the semester deadline.

9.1 Final Checklist of Project Achievements

The following table summarizes the project's adherence to the core criteria of the marking rubric (CLO 3):

Rubric Criteria	Evidence of Fulfillment	Status
Jira Project Setup (15m)	Scrum board (SKILL), Custom Workflow (In Review/PR), and Epic categorization.	COMPLETE
Planning & Scheduling (15m)	12-week Roadmap, Sprint 2 Goals, and equal Story Point distribution across team.	COMPLETE
Task Management (15m)	Active board movement, @mention comments, and daily status updates documented in Jira history.	COMPLETE
Risk & Quality (10m)	Risk Register (Assigned to Maaz), Mitigation Plans for MongoDB latency, and Acceptance Criteria on all stories.	COMPLETE
Innovation/Professionalism (5m)	Specialized "Risk" labels, WBS decomposition (SCRUM-16, 17, 18), and real-world relevance of the platform.	COMPLETE

Figure 7: Final Marking Rubric for CLO 3, serving as the benchmark for this project's successful implementation and demonstrating full compliance.

Figure 14: Final feature fulfillment verification, showing the fully functional platform meeting all essential requirements.

Ultimately, the SkillSwap project delivered more than a software prototype; it established a blueprint for student-led software development, characterized by professional accountability and technical excellence.